Definición del Proyecto - Etapa 1

Byron Thomas Oñate Delgado, Rodrigo Ignacio Sances Nuñez

1. Introducción

En la era digital actual, el acceso a la información ha crecido exponencialmente, pero esta abundancia ha traído consigo un desafío significativo: la dispersión de los recursos. Actualmente, la gestión de contenidos académicos y literarios carece de una centralización eficiente. Estudiantes universitarios, investigadores y lectores asiduos se enfrentan a diario con el problema de perder tiempo valioso intentando localizar textos específicos, organizar sus lecturas o recordar el material que ya han consultado. Esta fragmentación de la información no solo disminuye la productividad, sino que también genera frustración en usuarios que necesitan herramientas ágiles para gestionar sus bibliotecas digitales.

Frente a esta problemática, nuestro objetivo de negocio es proveer un repositorio centralizado, dinámico y escalable que optimice los tiempos de búsqueda, clasificación y retención de usuarios. Para lograr esto, nuestra propuesta de solución consiste en el desarrollo de 'Lector+', una aplicación web robusta diseñada bajo una arquitectura de API REST interoperable y respaldada por una base de datos relacional MySQL. Esta plataforma permitirá a los usuarios realizar búsquedas avanzadas mediante filtros precisos y gestionar colecciones personales de contenido, asegurando una experiencia fluida, escalable y preparada para futuras integraciones tecnológicas.

2. Tecnologías Utilizadas

El desarrollo de la plataforma Lector+ se sustenta en el siguiente stack tecnológico:

- Frontend (Capa de Presentación): HTML5, CSS3 y JavaScript Vanilla. Gestiona la maquetación responsiva, la interacción del usuario y el consumo asíncrono de los servicios web (Fetch API).
- Backend (Lógica de Negocio): Python 3 con el micro-framework Flask. Define una API RESTful estructurada en endpoints, que procesa las reglas de negocio y asegura la escalabilidad e interoperabilidad a través de JSON.
- Base de Datos (Persistencia): MySQL como motor relacional transaccional, garantizando la consistencia de los datos mediante el cumplimiento de las propiedades ACID.

- Despliegue y Control de Versiones: Git y GitHub para la administración y control del código fuente, junto a Render para el Cloud Hosting del backend.

3. Requerimientos y Casos de Uso

3.1 Requerimientos Funcionales (RF)

RF01 — Búsqueda y filtrado avanzado de contenidos

El sistema deberá permitir a los usuarios buscar material bibliográfico aplicando filtros obligatorios (tipo de contenido y etiqueta) y filtros opcionales (autor, fecha y orden de publicación). Los resultados obtenidos deberán mostrarse en una grilla estructurada para facilitar su revisión.

RF02 — Gestión de colecciones personales

El sistema deberá permitir a los lectores registrados agregar o remover contenidos de su biblioteca personal. Al confirmar la acción de guardado, el sistema deberá vincular al usuario con el documento y actualizar automáticamente la métrica de popularidad del material.

RF03 — Gestión y mantenedor de cuentas de usuario

El sistema deberá proveer un módulo de administración que permita al personal autorizado crear, visualizar, modificar y desactivar (mediante borrado lógico) perfiles de usuario, gestionando sus credenciales de acceso y roles dentro de la plataforma.

RF04 — Gestión y mantenedor del catálogo de contenidos

El sistema deberá incluir un módulo administrativo (CRUD) dedicado al registro, edición, categorización mediante etiquetas y eliminación lógica de todo el material bibliográfico y académico disponible en la plataforma.

RF05 — Autenticación y control de acceso por roles

El sistema deberá requerir credenciales válidas (correo electrónico y contraseña) para permitir el ingreso a la plataforma. Además, deberá validar el rol del usuario autenticado, restringiendo el acceso a los módulos de mantenedores exclusivamente a los perfiles con rol de Administrador.

3.2 Requerimientos No Funcionales (RNF)

RNF01 — Rendimiento de búsqueda

El sistema deberá mostrar los resultados de una búsqueda de contenido en un tiempo máximo de 2 segundos, considerando una base de datos de hasta 10.000 contenidos registrados.

RNF02 — Seguridad de contraseñas

El sistema deberá almacenar las contraseñas utilizando un algoritmo de hash seguro, como bcrypt o Argon2. Las contraseñas nunca deberán guardarse ni mostrarse como texto plano.

RNF03 — Control de acceso por roles

El sistema deberá controlar el acceso mediante los roles Lector y Administrador. Los lectores podrán gestionar únicamente su colección personal, mientras que solo los administradores podrán crear, modificar o desactivar usuarios, contenidos y etiquetas.

RNF04 — Integridad de las operaciones

Al agregar o eliminar un contenido de una colección, todas las modificaciones relacionadas deberán ejecutarse dentro de una transacción de base de datos. Si alguna operación falla, el sistema deberá revertir todos los cambios para evitar información incompleta o inconsistente.

RNF05 — Interoperabilidad de la API

La comunicación entre la interfaz web y el servidor deberá realizarse mediante una API REST, utilizando el formato JSON, el protocolo HTTPS y los métodos HTTP correspondientes, como GET, POST, PUT y DELETE.

RNF06 — Privacidad de las colecciones

El sistema deberá garantizar que cada lector solamente pueda consultar y modificar su propia colección personal. Los intentos de acceso a colecciones de otros usuarios deberán ser rechazados, excepto cuando un administrador cuente con autorización para ello.

4. Descripción de Casos de Uso

CU01: Consultar Contenido (Buscador)

Actor principal: Lector (Usuario Registrado).
Descripción: Permite al usuario buscar material bibliográfico o académico utilizando criterios específicos para acotar los resultados.
Flujo principal:
1. El usuario ingresa a la vista del buscador.
2. El sistema despliega los filtros obligatorios (Tipo, Etiqueta).
3. El usuario selecciona los parámetros deseados y presiona 'Buscar'.
4. El sistema valida los datos y consulta la base de datos a través de la API.
5. El sistema retorna una grilla paginada con los contenidos que coinciden con los filtros.

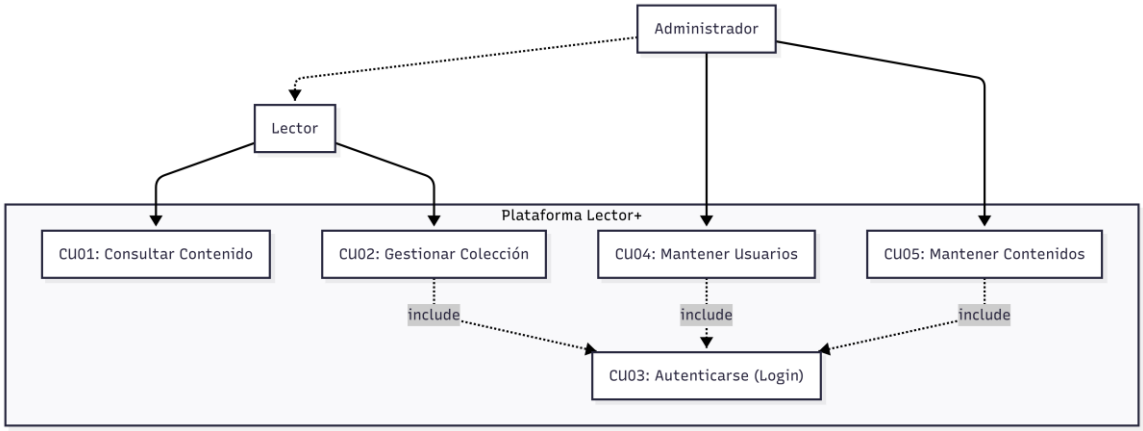
CU02: Gestionar Colección Personal (Transaccional)

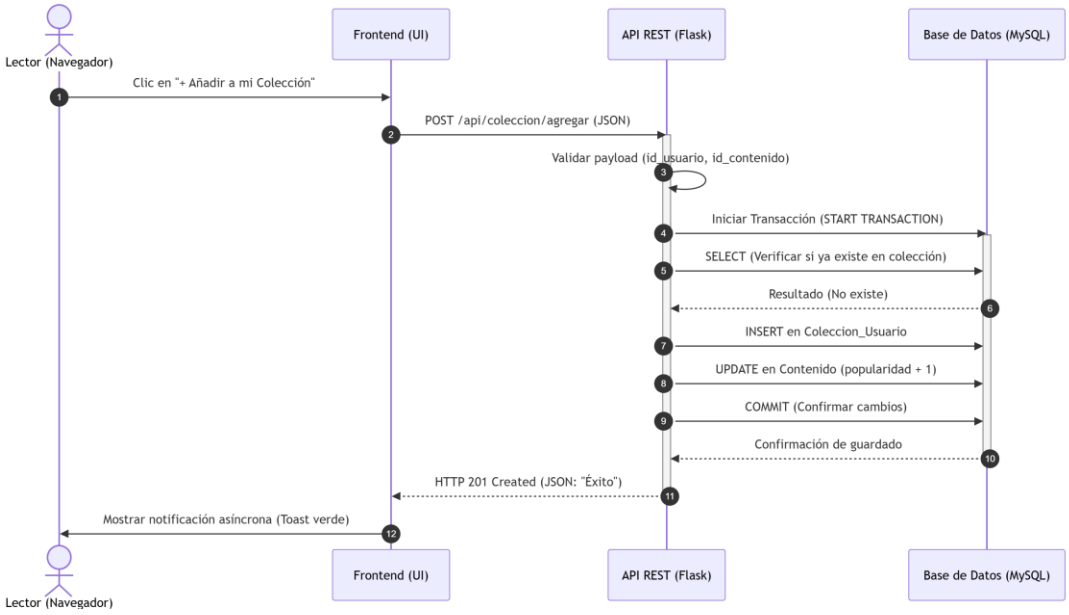
Actor principal: Lector (Usuario Registrado).
Descripción: Permite al lector guardar un contenido específico en su biblioteca personal

para acceso futuro.
Flujo principal:
1. El usuario visualiza el detalle de un contenido seleccionado.
2. El usuario presiona el botón '+ Añadir a mi Colección'.
3. El sistema (Backend) verifica que el contenido no exista previamente en la colección del usuario.
4. El sistema inserta el registro transaccional vinculando al usuario con el contenido y aumenta el contador de popularidad del material.
5. El sistema muestra una notificación asíncrona de éxito (toast) sin recargar la página.

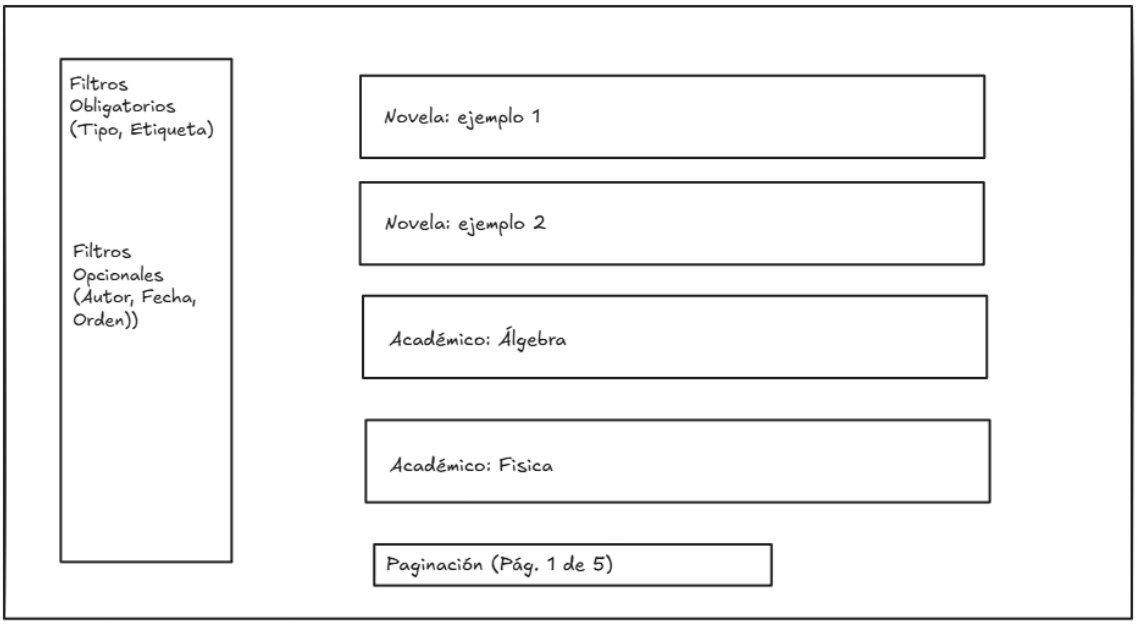
CU03: Mantener Usuarios y Contenidos (Mantenedores)

Actor principal: Administrador.
Descripción: Permite al personal administrativo crear, leer, actualizar o eliminar (CRUD) registros base del sistema.
Flujo principal:
1. El administrador inicia sesión y accede al panel de control.
2. Selecciona la entidad a gestionar (Usuarios o Contenido).
3. El sistema despliega el formulario correspondiente.
4. El administrador envía la petición de modificación y el sistema actualiza la base de datos relacional protegiendo la integridad de los datos.





5. Prototipos de Interfaz

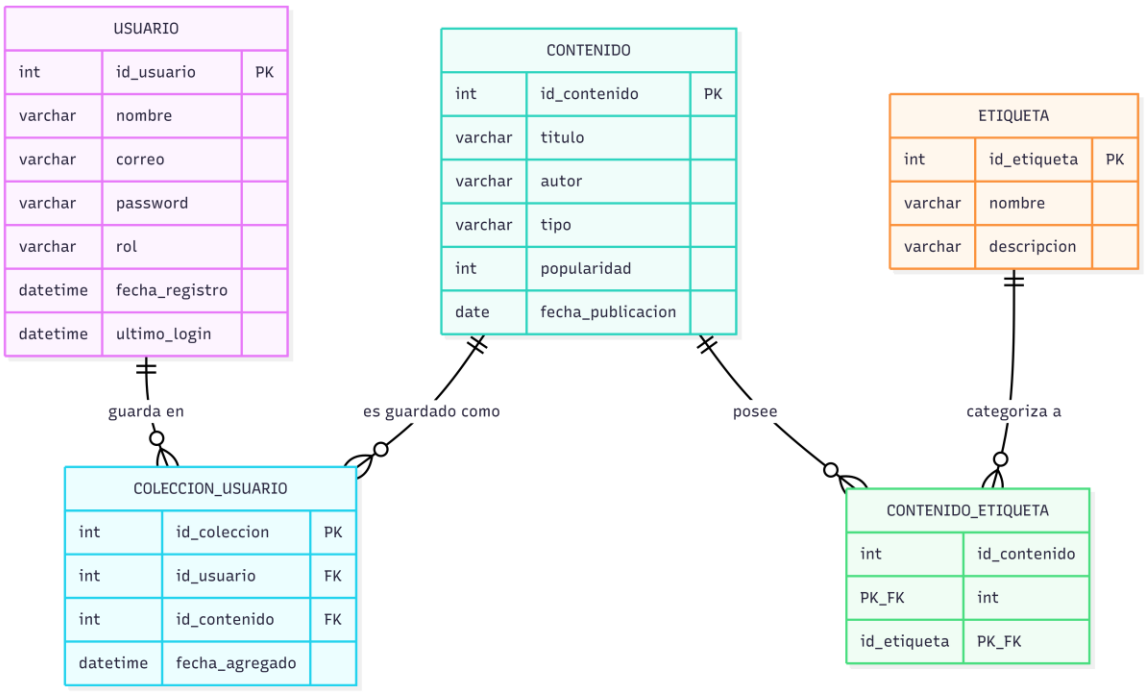




6. Modelo de Datos y Diccionario

6.1 Modelo Entidad-Relación

El Modelo ER define las entidades fuertes y sus interacciones a un nivel conceptual de negocio. Se identificaron tres entidades principales: Usuario, Contenido y Etiqueta. Para resolver las cardinalidades de 'Muchos a Muchos' (N:M) detectadas entre Lectores y Contenidos (Colecciones) y entre Contenidos y Etiquetas, se aplicó la normalización originando dos entidades asociativas.



Entidad: Usuario

- id_usuario: INT, Clave Primaria (PK), Autoincremental. Identificador único del usuario.
- nombre: VARCHAR(100), NOT NULL. Nombre completo o alias del lector.
- correo: VARCHAR(150), UNIQUE, NOT NULL. Correo electrónico para autenticación.
- password: VARCHAR(255), NOT NULL. Contraseña encriptada (Hash).
- rol: VARCHAR(20), DEFAULT 'Lector'. Nivel de acceso (Lector, Administrador).
- fecha_registro: DATETIME, DEFAULT CURRENT_TIMESTAMP. Trazabilidad de creación.
- ultimo_login: DATETIME, NULL. Registro de última actividad.

Entidad: Contenido

- id_contenido: INT, Clave Primaria (PK), Autoincremental. Identificador del material.
- titulo: VARCHAR(200), NOT NULL. Nombre de la obra o artículo.
- autor: VARCHAR(150), NOT NULL. Creador del contenido.
- tipo: VARCHAR(50), NOT NULL. Clasificación principal (ej. Novela, Académico).
- popularidad: INT, DEFAULT 0. Contador de cuántas veces ha sido guardado.
- fecha_publicacion: DATE, NULL. Fecha de emisión original de la obra.

Entidad: Etiqueta

- id_etiqueta: INT, Clave Primaria (PK), Autoincremental.
- nombre: VARCHAR(50), UNIQUE, NOT NULL. Nombre del descriptor (ej. "Ingeniería", "Fantasía").
- descripcion: VARCHAR(255), NULL. Explicación del alcance de la etiqueta.

Entidad Asociativa: Contenido_Etiqueta

- id_contenido: INT, PK Compuesta y FK hacia Contenido.
- id_etiqueta: INT, PK Compuesta y FK hacia Etiqueta.

Entidad Asociativa: Coleccion_Usuario

- id_coleccion: INT, Clave Primaria (PK), Autoincremental.
- id_usuario: INT, FK hacia Usuario.
- id_contenido: INT, FK hacia Contenido.
- fecha_agregado: DATETIME, DEFAULT CURRENT_TIMESTAMP. Auditoría de la transacción.

6.2 Modelo Relacional (Lógico)

La traducción del modelo conceptual al esquema lógico de base de datos relacional se define a continuación. (Leyenda: PK = Clave Primaria subrayada, FK = Clave Foránea en cursiva).

1. USUARIO (id_usuario (PK), nombre, correo, password, rol, fecha_registro, estado)

2. CONTENIDO (id_contenido (PK), titulo, autor, tipo, popularidad, fecha_publicacion)

3. ETIQUETA (id_etiqueta (PK), nombre, descripcion)

4. CONTENIDO_ETIQUETA (id_contenido (PK, FK), id_etiqueta (PK, FK))

5. COLECCION_USUARIO (id_coleccion (PK), id_usuario (FK), id_contenido (FK), fecha_agregado)

7. Consultas SQL Obligatorias y Álgebra Relacional

A continuación se presentan las 13 sentencias requeridas para la manipulación y definición de la base de datos (DML y DDL).

Consultas de Inserción (3 INSERT)

```
-- 1. Registro de un nuevo lector
INSERT INTO Usuario (nombre, correo, password, rol) VALUES ('Juan Pérez',
'juan@email.com', 'hash_seguro_123', 'Lector');

-- 2. Ingreso de un nuevo material académico
INSERT INTO Contenido (titulo, autor, tipo, popularidad, fecha_publicacion) VALUES
('Álgebra Lineal y sus Aplicaciones', 'David C. Lay', 'Académico', 0, '2015-05-20');

-- 3. Transacción: El lector guarda un libro en su colección
INSERT INTO Coleccion_Usuario (id_usuario, id_contenido, fecha_agregado) VALUES (1, 1,
NOW());
```

Consultas de Actualización (2 UPDATE)

```
-- 4. Aumentar la popularidad de un contenido al ser guardado
UPDATE Contenido SET popularidad = popularidad + 1 WHERE id_contenido = 1;

-- 5. Actualizar la fecha del último acceso de un usuario
UPDATE Usuario SET ultimo_login = NOW() WHERE id_usuario = 1;
```

Consultas de Eliminación (2 DELETE)

```
-- 6. El usuario remueve un contenido de su colección personal
DELETE FROM Coleccion_Usuario WHERE id_usuario = 1 AND id_contenido = 1;

-- 7. Se elimina una categorización errónea de un contenido
DELETE FROM Contenido_Etiqueta WHERE id_contenido = 1 AND id_etiqueta = 5;
```

Modificación de Estructura (2 ALTER)

```
-- 8. Se añade la columna 'estado' para implementar el borrado lógico
ALTER TABLE Usuario ADD estado BOOLEAN DEFAULT TRUE;

-- 9. Se amplía la capacidad del título para soportar nombres largos
ALTER TABLE Contenido MODIFY titulo VARCHAR(255) NOT NULL;
```

Eliminación de Estructura (1 DROP)

```
-- 10. Se elimina un índice que estaba consumiendo memoria innecesaria
DROP INDEX idx_fecha ON Contenido;
```

Consultas de Selección y Álgebra Relacional (3 SELECT)

```
11. Búsqueda de contenido por Etiqueta específica
SELECT c.titulo, c.autor, e.nombre AS categoria
FROM Contenido c
INNER JOIN Contenido_Etiqueta ce ON c.id_contenido = ce.id_contenido
INNER JOIN Etiqueta e ON ce.id_etiqueta = e.id_etiqueta
WHERE e.nombre = 'Ingeniería';
```

Álgebra Relacional: Proyección de título, autor y categoría de la Selección (nombre='Ingeniería') de la Reunión Natural de las tres tablas.

```
12. Visualización de la colección personal de un lector
SELECT u.nombre AS lector, c.titulo, cu.fecha_agregado
FROM Usuario u
INNER JOIN Coleccion_Usuario cu ON u.id_usuario = cu.id_usuario
INNER JOIN Contenido c ON cu.id_contenido = c.id_contenido
WHERE u.id_usuario = 1;
```

Álgebra Relacional: Proyección de lector, título y fecha de la Selección (id_usuario=1) de la Reunión Natural de las tres tablas.

13. Reporte de contenidos más populares para recomendaciones
SELECT titulo, tipo, popularidad
FROM Contenido
WHERE popularidad > 50
ORDER BY popularidad DESC;

Álgebra Relacional: Ordenamiento descendente por popularidad de la Proyección de título, tipo y popularidad de la Selección (popularidad > 50).

8. Repositorio de Código y Despliegue

https://github.com/byrononate714-hue/plataforma-lector-api

https://plataforma-lector-api.onrender.com